

**DATABASE MANAGEMENT SYSTEM
PROJECT REPORT**

**TOPIC 31:- DESIGN AND IMPLEMENTATION OF A DATABASE MANAGEMENT SYSTEM
FOR EVENT REGISTRATION AND VENUE SCHEDULING SYSTEM**



Course: Database Management Systems (DBMS)

University: Woxsen University

Academic Year: 2025-2029

Faculty: Dr. Kiran Mayee Adavala

TEAM MEMBERS

1. Ananey Sharma - 25WU0102017
2. Rudra Kumar - 25WU0101113
3. Prachi Singhania - 25WU0101099
4. Mithil D. Parikh - 25WU0101072

Table of Contents

1. Abstract
2. Introduction and Problem Statement
3. Objectives, Scope and Users
4. Functional and Non-Functional Requirements
5. Software and Hardware Requirements
6. System Architecture and Workflow
7. ER Diagram and Relationship Analysis
8. Relational Schema and Normalization
9. Data Dictionary
10. SQL Commands Used
11. Queries and Reports
12. UI Design and Screenshots
13. Implementation Details
14. Testing and Validation
15. Security, Integrity and Performance Considerations
16. Conclusion and Future Enhancements
17. References
18. Contribution of Each Member
19. Appendix: GitHub Repository

1. Abstract

The Event Registration and Venue Scheduling System is a relational database management system developed for organizing and managing campus events through a centralized database. The system stores event categories, organizing entities, venues, schedules, participants and registrations as related records rather than as isolated files or manual entries.

The implementation contains seven principal relations: EVENT_CATEGORIES, ORGANIZERS, VENUES, EVENTS, EVENT_SCHEDULES, PARTICIPANTS and REGISTRATIONS. EVENTS acts as the central operational entity, while REGISTRATIONS acts as an associative relation between EVENTS and PARTICIPANTS. Primary keys, foreign keys, UNIQUE constraints, CHECK conditions and default values are used to maintain data integrity.

A web-based interface provides an administrative dashboard, event management, participant management, registration monitoring, venue scheduling, organizer/category management, a public student portal and an academic SQL query console. The project demonstrates conceptual design, relational mapping, normalization up to 3NF, SQL implementation, database connectivity, CRUD-oriented interaction and testing.

2. Introduction and Problem Statement

2.1 Background

Universities conduct a large number of academic, technical, cultural and sports activities. An event requires coordinated information about its category, organizing entity, venue, date, time, capacity and participants. As the number of events increases, maintaining this information manually can lead to duplication, inconsistent records and difficulty in retrieving complete event information.

2.2 Problem Statement

The problem addressed by this project is the absence of a single structured database for managing event registration and venue scheduling. Without a centralized relational system, organizers may find it difficult to coordinate venue allocations, participants may not have a clear event catalogue, and administrators may struggle to generate registration, payment and scheduling reports.

- Event, organizer and category information can become duplicated across records.
- Venue allocation and event timing need to be tracked together.
- Participant registrations must be associated with the correct event.
- Duplicate registrations should be prevented.
- Registration and payment statuses need to be visible to administrators.
- Management needs consolidated reports instead of manually combining records.

2.3 Proposed Solution

The proposed solution is a normalized MySQL/MariaDB database connected to a web application. The database stores each major entity once and connects related entities using foreign keys. The UI provides controlled forms and views for maintaining these records and presents analytical information through the dashboard.

3. Objectives, Scope and Users

3.1 Objectives

- Design a normalized relational database for campus event registration and venue scheduling.
- Represent event categories, organizers, venues, events, schedules, participants and registrations.
- Implement primary keys, foreign keys and domain constraints.
- Support creation, viewing, updating and deletion of relevant records.
- Prevent duplicate participant-event registrations.
- Provide SQL queries using joins, aggregation, grouping and subqueries.
- Provide a practical web interface connected to the database.
- Generate useful operational information such as schedules, registrations and revenue.

3.2 Scope

- Event creation and management.
- Event category and organizer management.
- Venue information and schedule allocation.
- Participant/student directory management.
- Registration and payment-status tracking.
- Public event browsing and registration.
- Dashboard analytics and SQL reporting.
- Database schema visualization.

3.3 Intended Users

User	Main Activities
Administrator	Manage events, venues, organizers, categories, participants and registrations; view reports.
Organizer/Club	Create and manage events and view scheduling information.
Student/Participant	Browse upcoming events and register.
Faculty/Evaluator	Review database design, SQL, UI and project implementation.

4. Functional and Non-Functional Requirements

4.1 Functional Requirements

- Create, view, update and delete events.
- Assign an event category and organizer.
- Create and maintain venue records.
- Schedule an event using date, start time, end time and venue.
- Register participants for events.
- Maintain registration and payment status.
- Search/filter events and participants.
- Display event capacity and enrollment information.
- Display organizer/category statistics.

- Execute predefined academic SQL queries.

4.2 Non-Functional Requirements

- Data consistency through relational constraints.
- Usable interface with clear modules and forms.
- Fast retrieval through indexed/unique searchable attributes where applicable.
- Maintainable separation between database, application logic and presentation.
- Validation of user input before database operations.
- Scalable relational design that can accommodate additional event records.

5. Software and Hardware Requirements

Component	Requirement
Database	MariaDB/MySQL (interface indicates MariaDB 10.4)
Database Administration	phpMyAdmin
Web Server	Apache through XAMPP
Backend	PHP
Frontend	HTML, CSS, JavaScript
IDE	Visual Studio Code
ER Diagram	diagrams.net / draw.io
Version Control	Git and GitHub
Browser	Modern web browser
Hardware	64-bit computer, 4 GB+ RAM recommended, adequate storage and display

6. System Architecture and Workflow

6.1 Three-Layer View

Layer	Components	Purpose
Presentation Layer	HTML, CSS, JavaScript	Forms, dashboard, tables, filters and user interaction.
Application Layer	PHP	Validation, CRUD operations, query execution and business logic.
Data Layer	MariaDB/MySQL	Persistent storage, relationships, constraints and SQL reporting.

6.2 Event Workflow

Step	Process
1	Administrator/organizer enters event details.
2	Event is associated with an existing category and organizer.
3	Venue and schedule are assigned through EVENT SCHEDULES.
4	Students view the event through the public portal.
5	Participant registration creates a REGISTRATIONS record.
6	Registration/payment status is maintained.
7	Dashboard and SQL reports summarize operational data.

7. ER Diagram and Relationship Analysis

The following ER diagram is the primary conceptual model for the project. It explicitly shows all seven entities, their principal attributes, primary/foreign keys and relationship cardinalities.

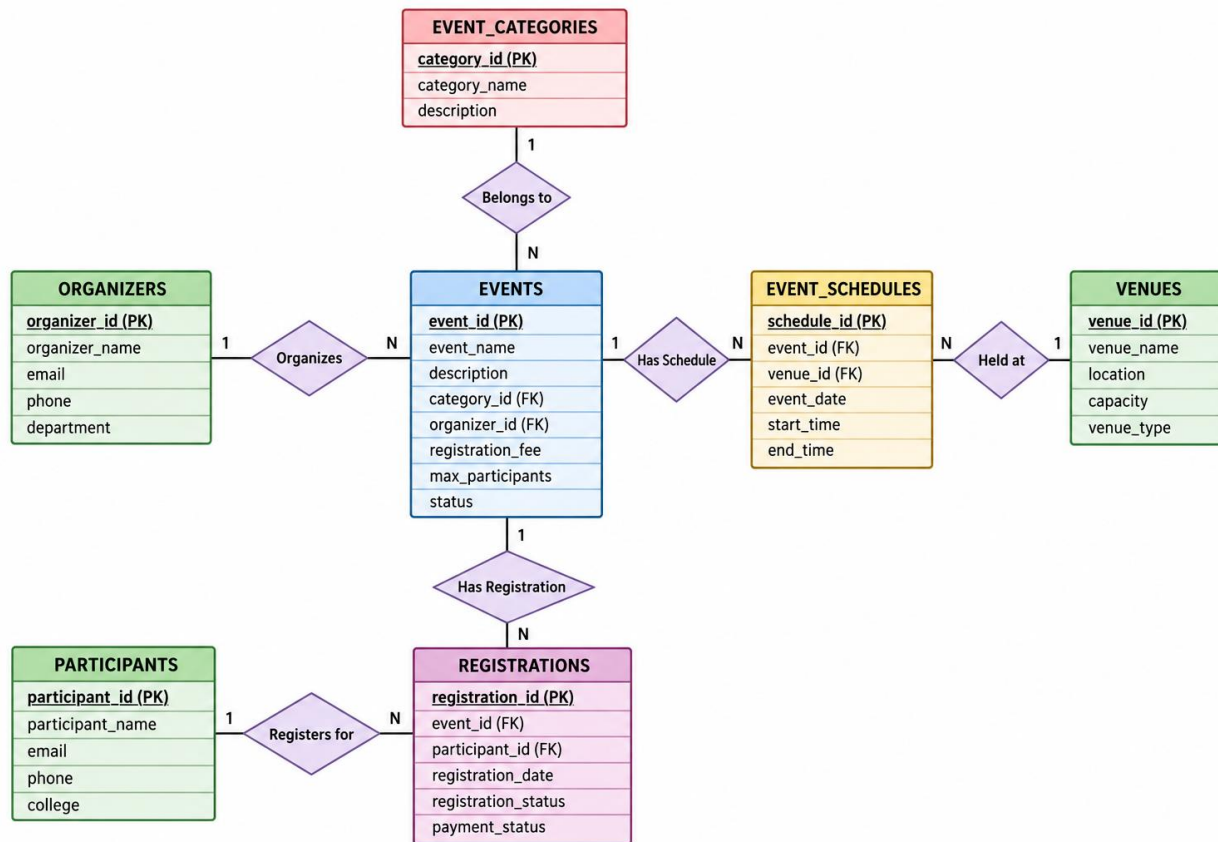


Figure 1. Final Entity-Relationship Diagram

7.1 Entity Description

Entity	Purpose
EVENT_CATEGORIES	Stores the classification of events.
ORGANIZERS	Stores clubs/organizing entities and contact/department information.
EVENTS	Central entity storing event details, category, organizer, fee, capacity and status.
EVENT_SCHEDULES	Associates events with venues and stores date/time information.
VENUES	Stores venue name, location, capacity and venue type.
PARTICIPANTS	Stores students/participants who can register for events.
REGISTRATIONS	Associative entity linking participants to events and storing registration/payment status.

7.2 Relationships and Cardinality

Relationship	Cardinality	Interpretation
EVENT_CATEGORIES — EVENTS	1:N	One category can classify many events; each event belongs to a category.
ORGANIZERS — EVENTS	1:N	One organizer can organize many events; each event has an organizer.
EVENTS — EVENT SCHEDULES	1:N	An event can have schedule records.

VENUES — EVENT_SCHEDULES	1:N	A venue can host multiple scheduled events at different time slots.
EVENTS — REGISTRATIONS	1:N	An event can have many registrations.
PARTICIPANTS — REGISTRATIONS	1:N	A participant can have many registrations.
PARTICIPANTS — EVENTS	M:N	Resolved through REGISTRATIONS.

8. Relational Schema and Normalization

8.1 Relational Schema

EVENT_CATEGORIES(category_id PK, category_name UNIQUE, description)

ORGANIZERS(organizer_id PK, organizer_name, email UNIQUE, phone, department)

VENUES(venue_id PK, venue_name UNIQUE, location, capacity, venue_type)

EVENTS(event_id PK, event_name, description, category_id FK, organizer_id FK, registration_fee, max_participants, status)

EVENT_SCHEDULES(schedule_id PK, event_id FK, venue_id FK, event_date, start_time, end_time)

PARTICIPANTS(participant_id PK, participant_name, email UNIQUE, phone, college)

REGISTRATIONS(registration_id PK, event_id FK, participant_id FK, registration_date, registration_status, payment_status, UNIQUE(event_id, participant_id))

8.2 Normalization

The database is designed up to Third Normal Form (3NF). Normalization separates independent facts into appropriate relations and prevents unnecessary repetition.

- First Normal Form (1NF): each column stores atomic values and there are no repeating groups.
- Second Normal Form (2NF): attributes are dependent on the complete key of their relation. Event facts, participant facts and registration facts are separated into their respective relations.
- Third Normal Form (3NF): non-key attributes do not depend on other non-key attributes. Organizer, venue and category information are stored independently rather than repeatedly inside EVENTS.

8.3 Functional Dependency Examples

Relation	Representative Functional Dependencies
EVENT_CATEGORIES	category_id → category_name, description
ORGANIZERS	organizer_id → organizer_name, email, phone, department
VENUES	venue_id → venue_name, location, capacity, venue_type
PARTICIPANTS	participant_id → participant_name, email, phone, college
EVENTS	event_id → event_name, description, category_id, organizer_id, registration_fee, max_participants, status
EVENT_SCHEDULES	schedule_id → event_id, venue_id, event_date, start_time, end_time
REGISTRATIONS	registration_id → event_id, participant_id, registration_date, registration_status, payment_status

9. Data Dictionary

EVENT_CATEGORIES

Field	Data Type	Constraint	Description
category_id	INT AUTO INCREMENT	PK	Unique category ID
category_name	VARCHAR(100)	NOT NULL, UNIQUE	Category name
description	VARCHAR(255)	—	Category description

ORGANIZERS

Field	Data Type	Constraint	Description
organizer_id	INT AUTO INCREMENT	PK	Unique organizer ID
organizer_name	VARCHAR(100)	NOT NULL	Organizer/club name
email	VARCHAR(100)	UNIQUE	Email
phone	VARCHAR(15)	—	Phone
department	VARCHAR(100)	—	Department

VENUES

Field	Data Type	Constraint	Description
venue_id	INT AUTO INCREMENT	PK	Unique venue ID
venue_name	VARCHAR(100)	NOT NULL, UNIQUE	Venue name
location	VARCHAR(150)	—	Location
capacity	INT	CHECK > 0	Venue capacity
venue_type	VARCHAR(50)	—	Venue type

EVENTS

Field	Data Type	Constraint	Description
event_id	INT AUTO INCREMENT	PK	Unique event ID
event_name	VARCHAR(150)	NOT NULL	Event title
description	TEXT	—	Description
category_id	INT	FK	Event category
organizer_id	INT	FK	Organizer
registration_fee	DECIMAL(10,2)	DEFAULT 0, CHECK >= 0	Registration fee
max_participants	INT	CHECK > 0	Maximum participants
status	VARCHAR(20)	CHECK/DEFAULT	Event status

EVENT_SCHEDULES

Field	Data Type	Constraint	Description
schedule_id	INT AUTO INCREMENT	PK	Unique schedule ID
event_id	INT	FK	Scheduled event
venue_id	INT	FK	Assigned venue
event_date	DATE	NOT NULL	Event date
start_time	TIME	NOT NULL	Start time
end_time	TIME	CHECK > start_time	End time

PARTICIPANTS

Field	Data Type	Constraint	Description
participant_id	INT AUTO INCREMENT	PK	Unique participant ID
participant_name	VARCHAR(100)	NOT NULL	Participant name
email	VARCHAR(100)	UNIQUE	Participant email
phone	VARCHAR(15)	—	Phone
college	VARCHAR(150)	—	College/institution

REGISTRATIONS

Field	Data Type	Constraint	Description
-------	-----------	------------	-------------

registration_id	INT AUTO INCREMENT	PK	Unique registration ID
event_id	INT	FK	Registered event
participant_id	INT	FK	Participant
registration_date	DATE	DEFAULT	Registration date
registration_status	VARCHAR(20)	CHECK/DEFAULT	Registration status
payment_status	VARCHAR(20)	CHECK/DEFAULT	Payment status
event_id + participant_id	UNIQUE	Composite uniqueness	Prevents duplicate registration

10. SQL Commands Used

10.1 DDL

```
CREATE DATABASE IF NOT EXISTS event_management;
USE event_management;
```

```
CREATE TABLE event_categories (
    category_id INT AUTO_INCREMENT PRIMARY KEY,
    category_name VARCHAR(100) NOT NULL UNIQUE,
    description VARCHAR(255)
);
```

```
CREATE TABLE events (
    event_id INT AUTO_INCREMENT PRIMARY KEY,
    event_name VARCHAR(150) NOT NULL,
    description TEXT,
    category_id INT NOT NULL,
    organizer_id INT NOT NULL,
    registration_fee DECIMAL(10,2) DEFAULT 0.00,
    max_participants INT NOT NULL,
    status VARCHAR(20) DEFAULT 'Upcoming',
    FOREIGN KEY (category_id) REFERENCES event_categories(category_id),
    FOREIGN KEY (organizer_id) REFERENCES organizers(organizer_id)
);
```

10.2 DML

```
INSERT INTO events
(event_name, description, category_id, organizer_id,
registration_fee, max_participants, status)
VALUES ('AI and Machine Learning Workshop',
'Hands-on workshop covering fundamentals of AI and ML.',
1, 1, 0.00, 100, 'Upcoming');

UPDATE events
SET registration_fee = 250.00
WHERE event_id = 2;
```

```
DELETE FROM registrations
WHERE registration_id = 5;
```

10.3 Integrity Constraints

- Primary keys uniquely identify each record.
- Foreign keys maintain referential relationships.
- UNIQUE constraints prevent duplicate category names, organizer emails, venue names and participant emails.
- Composite UNIQUE(event_id, participant_id) prevents duplicate registration.
- CHECK constraints enforce valid capacity, fee, time and status values.
- DEFAULT values reduce missing status/date information.

11. Queries and Reports

11.1 Multi-table Join

```
SELECT          e.event_name,          c.category_name,          o.organizer_name,
e.registration_fee,          e.max_participants,          e.status
FROM          events          e
JOIN          event_categories          c          ON          e.category_id=c.category_id
JOIN organizers o ON e.organizer_id=o.organizer_id;
```

11.2 Registration Count

```
SELECT          e.event_name,          COUNT(r.registration_id)          AS          total_registrations
FROM          events          e
LEFT JOIN          registrations          r          ON          e.event_id=r.event_id
GROUP BY e.event_id, e.event_name;
```

11.3 Revenue Report

```
SELECT          e.event_name,
SUM(CASE          WHEN          r.payment_status='Paid'
THEN          e.registration_fee          ELSE          0          END)          AS          revenue
FROM          events          e
LEFT JOIN          registrations          r          ON          e.event_id=r.event_id
GROUP BY e.event_id, e.event_name;
```

11.4 Schedule Report

```
SELECT          e.event_name,          v.venue_name,          es.event_date,
es.start_time,          es.end_time
FROM          event_schedules          es
JOIN          events          e          ON          es.event_id=e.event_id
JOIN          venues          v          ON          es.venue_id=v.venue_id
ORDER BY es.event_date, es.start_time;
```

11.5 Above-Average Registration Subquery

```
SELECT      e.event_name,      COUNT(r.registration_id)      AS      registrations
FROM      events      e      JOIN      registrations      r      ON      e.event_id=r.event_id
GROUP      BY      e.event_id,      e.event_name
HAVING      COUNT(r.registration_id)      >=      (
SELECT      AVG(event_count)
FROM      (
SELECT      COUNT(*)      AS      event_count
FROM      registrations      GROUP      BY      event_id
)
);
```

The SQL console shown in the UI contains academic presets for multi-table joins, organizer analytics and revenue, venue capacity versus occupancy, above-average events, category revenue/average ticket price and participant demographic reach.

12. UI Design and Screenshots

The interface was implemented as a local web application. The following screenshots document the main functional screens supplied for this report.

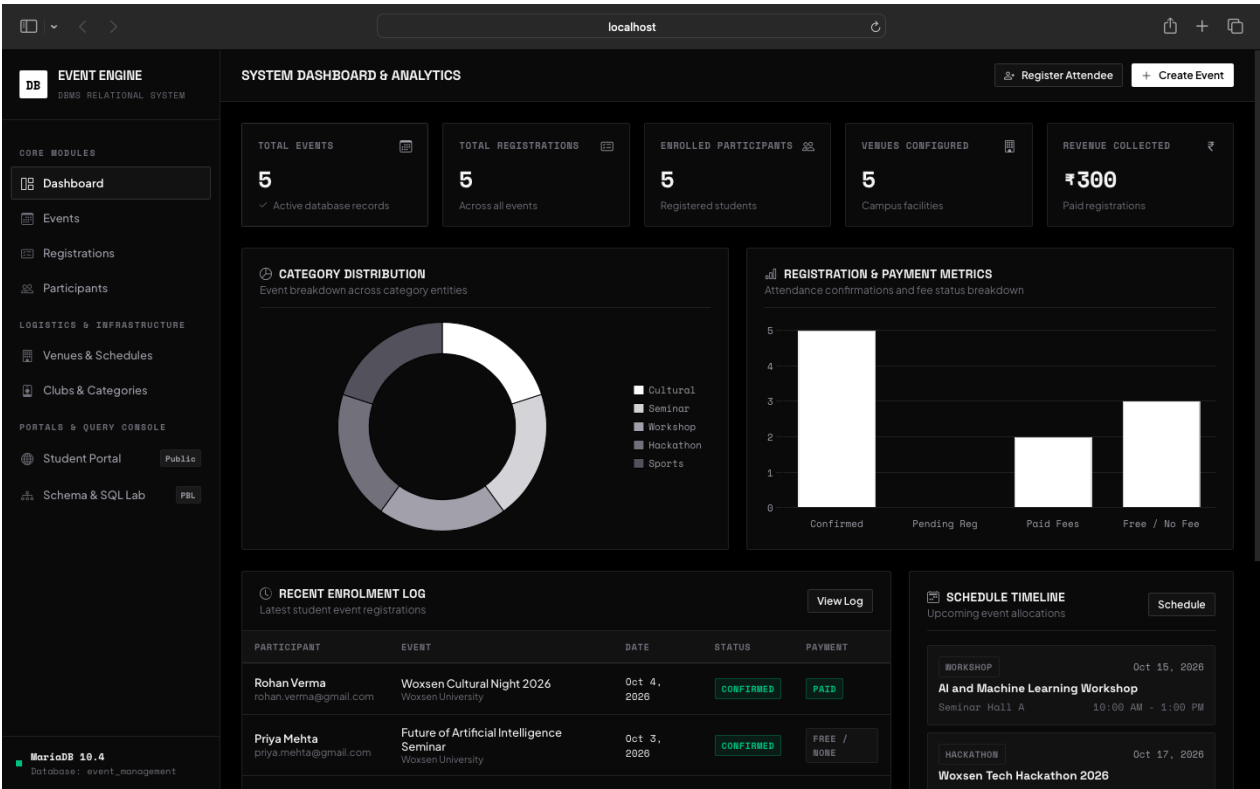


Figure 2. System Dashboard and Analytics

The dashboard summarizes total events, registrations, participants, venues and revenue, and provides category, payment and recent enrollment analytics.

The screenshot shows the 'CREATE EVENT' form in the Event Engine application. The form is a modal window with the following fields and options:

- EVENT TITLE ***: Text input field containing 'AI Innovation Workshop 2026'.
- DESCRIPTION**: Text area containing 'Brief event overview...'.
- CATEGORY ***: Dropdown menu showing '-- Choose Category --'.
- ORGANIZING ENTITY ***: Dropdown menu showing '-- Choose Organizer --'.
- REGISTRATION FEE (*) ***: Text input field containing '0.00'.
- MAX CAPACITY ***: Text input field containing '100'.
- STATUS ***: Dropdown menu showing 'Upcoming'.
- VENUE ALLOCATION & TIMING (OPTIONAL)**:
 - ASSIGNED VENUE**: Dropdown menu showing '-- Select Venue (Optional) --'.
 - DATE**: Text input field containing '06/10/2026'.
 - START TIME**: Text input field containing '12:30'.
 - END TIME**: Text input field containing '12:30'.

At the bottom of the form are two buttons: 'Cancel' and 'Save Event'.

Figure 3. Create Event Form

The Create Event form accepts event title, description, category, organizer, registration fee, maximum capacity, status and optional venue/timing details.

The screenshot shows the 'EVENTS DIRECTORY & MANAGEMENT' page in the Event Engine application. The page features a search bar and filters for categories and statuses. It displays a grid of event cards, each with the following information:

- Category**: SPORTS, CULTURAL, SEMINAR, HACKATHON, WORKSHOP.
- Event Title**: Inter-College Sports Meet 2026, Woxsen Cultural Night 2026, Future of Artificial Intelligence Seminar, Woxsen Tech Hackathon 2026, AI and Machine Learning Workshop.
- Description**: Brief description of the event.
- Organized by**: Sports Club, Cultural Club, Innovation Cell, Tech Club.
- Venue**: University Sports Ground, Main Auditorium, Seminar Hall B, Main Auditorium, Seminar Hall A.
- Schedule**: Date and time of the event.
- Enrollment**: Progress bar showing enrolled participants (e.g., 0 / 300, 1 / 500, 1 / 200, 1 / 150, 2 / 100).
- Status**: UPCOMING, FREE.
- Actions**: Details, Edit, Delete buttons.

Figure 4. Events Directory and Management

The Events Directory supports searching/filtering and presents event cards with category, fee, organizer, venue, schedule, enrollment and edit/delete actions.

EVENT ENGINE

DBMS RELATIONAL SYSTEM

CORE MODULES

Dashboard

Events

Registrations

Participants

LOGISTICS & INFRASTRUCTURE

Venues & Schedules

Clubs & Categories

PORTALS & QUERY CONSOLE

Student Portal

Schema & SQL Lab

MariaDB 10.4

Database: event_management

localhost

PARTICIPANT REGISTRATIONS LOG

Register Attendee

Create Event

Search by participant name, email, college or event...

All Registration Statuses

All Payment Statuses

New Registration

REG ID	STUDENT INFO	EVENT	FEE	REGISTRATION STATUS	PAYMENT STATUS	ACTIONS
#5	Rohan Verma rohan.verma@gmail.com Woxsen University	Woxsen Cultural Night 2026 Cultural	₹100.00	CONFIRMED	PAID	
#4	Priya Mehta priya.mehta@gmail.com Woxsen University	Future of Artificial Intelligence Seminar Seminar	Free	CONFIRMED	FREE / NONE	
#3	Arjun Kumar arjun.kumar@gmail.com Woxsen University	Woxsen Tech Hackathon 2026 Hackathon	₹200.00	CONFIRMED	PAID	
#2	Ananya Singh ananya.singh@gmail.com Woxsen University	AI and Machine Learning Workshop Workshop	Free	CONFIRMED	FREE / NONE	
#1	Rahul Sharma rahul.sharma@gmail.com Woxsen University	AI and Machine Learning Workshop Workshop	Free	CONFIRMED	FREE / NONE	

Figure 5. Participant Registrations Log

The Registration Log displays registration ID, participant details, event, fee, registration status, payment status and deletion controls.

EVENT ENGINE

DBMS RELATIONAL SYSTEM

CORE MODULES

Dashboard

Events

Registrations

Participants

LOGISTICS & INFRASTRUCTURE

Venues & Schedules

Clubs & Categories

PORTALS & QUERY CONSOLE

Student Portal

Schema & SQL Lab

MariaDB 10.4

Database: event_management

REGISTERED STUDENT DIRECTORY

Register Attendee

Create Event

Search by student name, email, phone or college...

Add Student

ID	STUDENT NAME	PHONE	COLLEGE / INSTITUTION	ENROLLMENTS	ACTIONS
#5	Rohan Verma rohan.verma@gmail.com	9876510065	WOXSEN UNIVERSITY	1 REGISTERED (1 CONFIRMED)	
#4	Priya Mehta priya.mehta@gmail.com	9876510064	WOXSEN UNIVERSITY	1 REGISTERED (1 CONFIRMED)	
#3	Arjun Kumar arjun.kumar@gmail.com	9876510063	WOXSEN UNIVERSITY	1 REGISTERED (1 CONFIRMED)	
#2	Ananya Singh ananya.singh@gmail.com	9876510062	WOXSEN UNIVERSITY	1 REGISTERED (1 CONFIRMED)	
#1	Rahul Sharma rahul.sharma@gmail.com	9876510061	WOXSEN UNIVERSITY	1 REGISTERED (1 CONFIRMED)	

MariaDB 10.4

Database: event_management

Figure 6. Registered Student Directory

The Participant Directory displays participant contact information, institution, enrollment counts and management actions.

EVENT ENGINE

DBMS RELATIONAL SYSTEM

CORE MODULES

Dashboard

Events

Registrations

Participants

LOGISTICS & INFRASTRUCTURE

Venues & Schedules

Clubs & Categories

PORTALS & QUERY CONSOLE

Student Portal

Schema & SQL Lab

MariaDB 10.4

Database: event_management

VENUES & TIME SCHEDULES

Register Attendee

Create Event

COMPUTER LAB

CSE Lab 1

CSE Block

Capacity: 60

Scheduled: 0

Edit

AUDITORIUM

Main Auditorium

Main Academic Block

Capacity: 1000

Scheduled: 2

Edit

SEMINAR HALL

Seminar Hall A

Academic Block A

Capacity: 200

Scheduled: 1

Edit

SEMINAR HALL

Seminar Hall B

Academic Block B

Capacity: 150

Scheduled: 1

Edit

OPEN GROUND

University Sports Ground

Sports Complex

Capacity: 500

Scheduled: 1

Edit

EVENT SCHEDULE ALLOCATIONS

Time slots with automated venue overlap conflict checks

Add Schedule

SCHEDULE ID	EVENT NAME	VENUE & LOCATION	EVENT DATE	TIME SLOT	ACTIONS
#1	AI and Machine Learning Workshop Workshop • Tech Club	Seminar Hall A Academic Block A	OCT 15, 2026	10:00 AM - 1:00 PM	
#2	Woxsen Tech Hackathon 2026 Hackathon • Tech Club	Main Auditorium Main Academic Block	OCT 17, 2026	9:00 AM - 6:00 PM	
#3	Future of Artificial Intelligence Seminar Seminar • Innovation Cell	Seminar Hall B Academic Block B	OCT 20, 2026	11:00 AM - 2:00 PM	
#4	Woxsen Cultural Night 2026 Cultural • Cultural Club	Main Auditorium Main Academic Block	OCT 25, 2026	5:00 PM - 9:00 PM	
#5	Inter-College Sports Meet 2026 Sports • Sports Club	University Sports Ground Sports Complex	NOV 2, 2026	8:00 AM - 5:00 PM	

Figure 7. Venues and Time Schedules

The Venues & Time Schedules screen displays venue capacity and scheduled-event counts and lists schedule allocations.

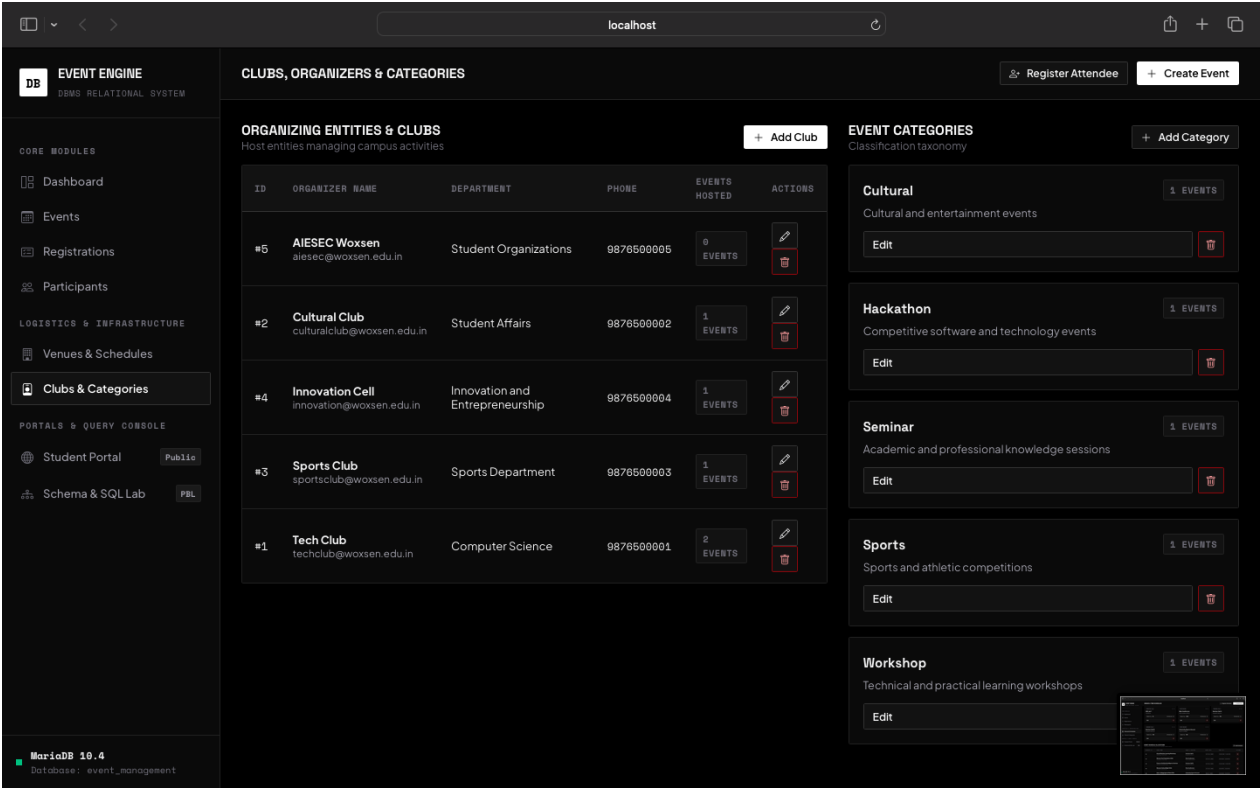


Figure 8. Clubs, Organizers and Event Categories

The Clubs, Organizers & Categories screen manages organizing entities and the event classification taxonomy.

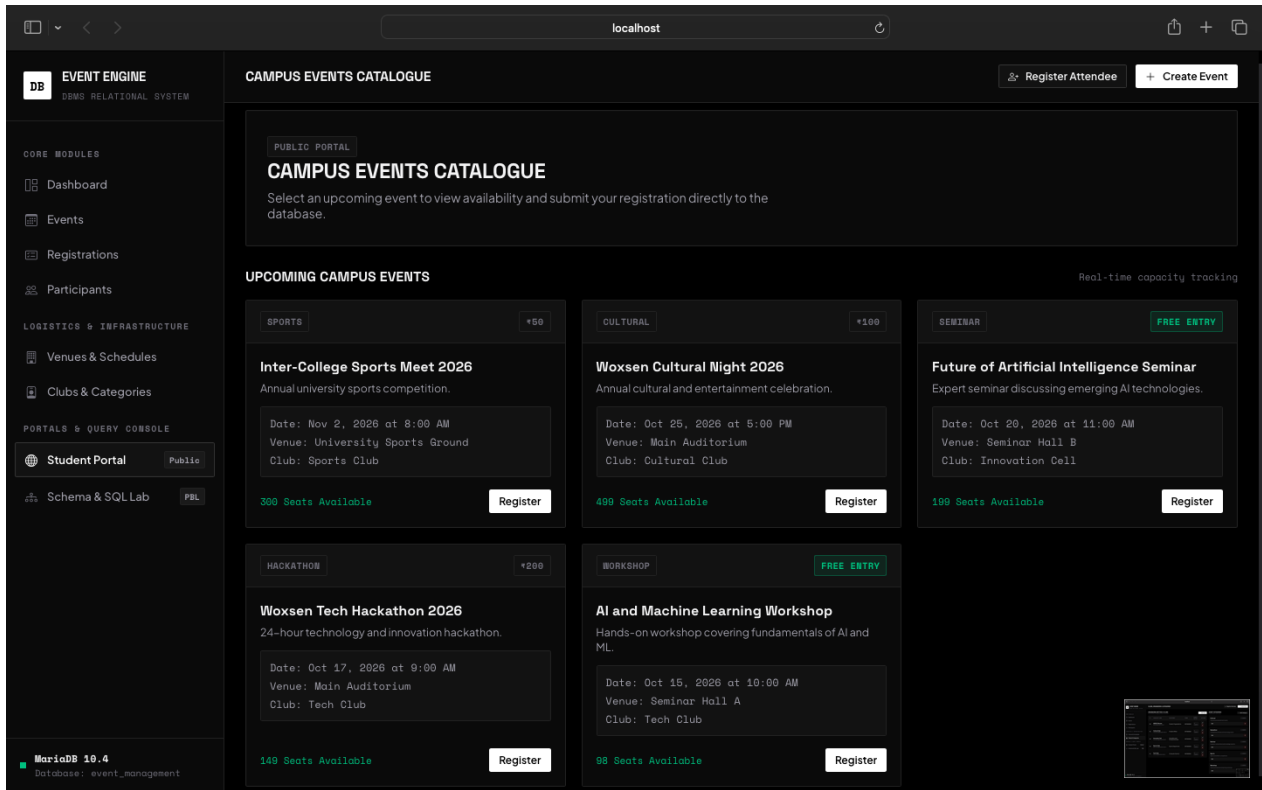


Figure 9. Public Student Event Portal

The Public Student Portal displays upcoming events, venue/date information, available seats and registration controls.

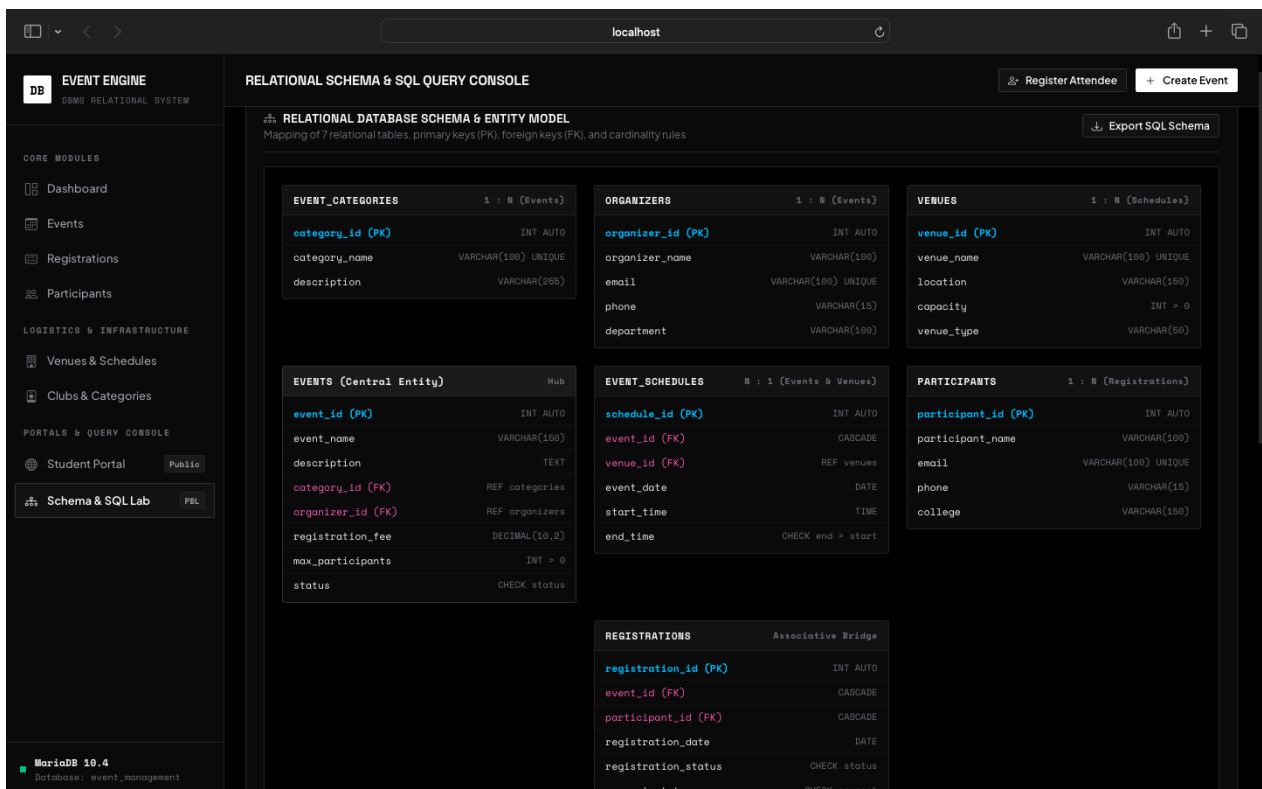


Figure 10. Relational Schema and Entity Model

The Relational Schema & SQL Lab screen visualizes the seven tables, keys, constraints and relationship structure.

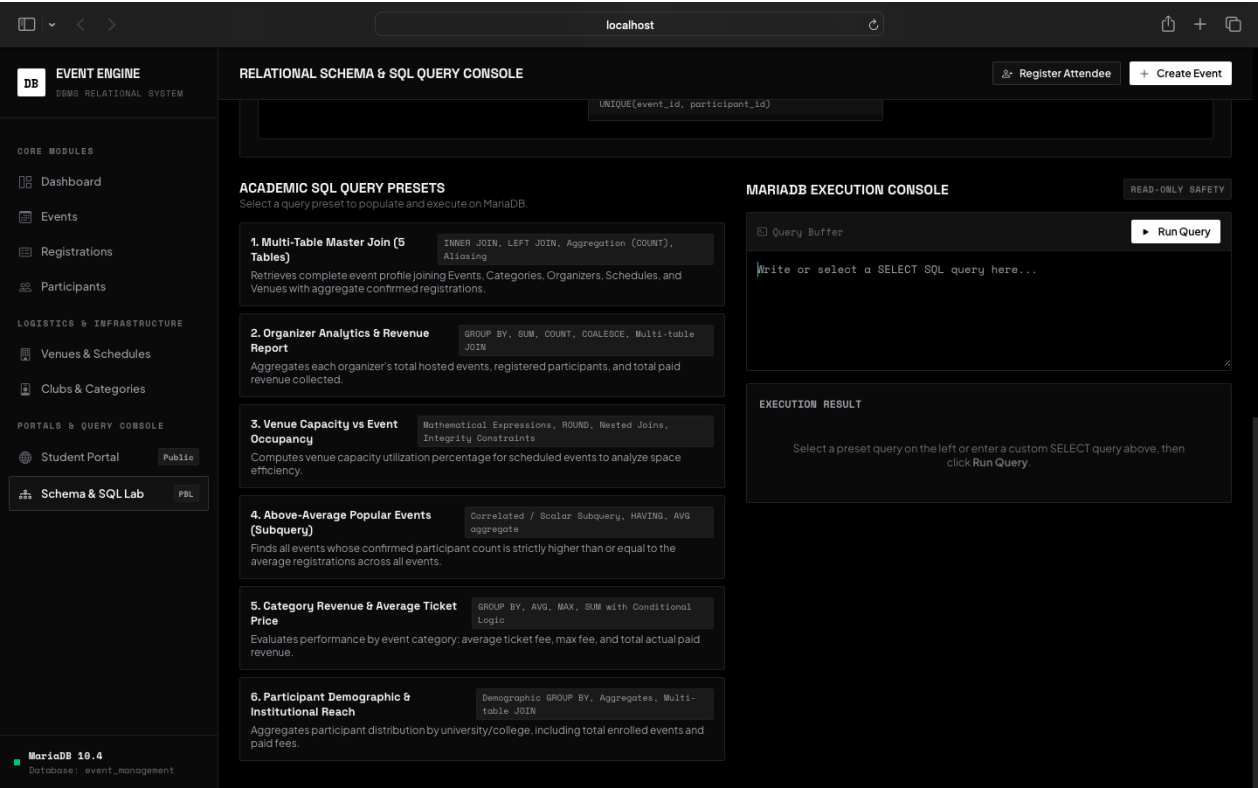


Figure 11. Academic SQL Query Console

The Academic SQL Query Console provides predefined reporting queries and a read-only SELECT execution area.

13. Implementation Details

13.1 Technology Stack

Layer	Technology	Purpose
Database	MariaDB/MySQL	Persistent relational data
Administration	phpMyAdmin	Database and SQL management
Server	Apache/XAMPP	Local application server
Backend	PHP	Database connectivity and application logic
Frontend	HTML/CSS/JavaScript	User interface and interactions
Development	Visual Studio Code	Source-code development
Version Control	Git/GitHub	Repository and contribution tracking

13.2 Database Connectivity

```
<?php
$conn = mysqli_connect("localhost","root","","event_management");
if (!$conn) {
    die("Database connection failed: " . mysqli_connect_error());
}
```

}
?>

The application connects to the event_management database and uses SQL queries to retrieve and manipulate records. Form values are mapped to the appropriate database entities rather than storing repeated entity information.

13.3 Major Modules

- Dashboard and Analytics
- Events Directory
- Create Event
- Participant Directory
- Registration Log
- Venues & Time Schedules
- Clubs/Organizers & Categories
- Public Student Portal
- Relational Schema & SQL Query Console

14. Testing and Validation

ID	Test Case	Expected Result	Status
T01	Dashboard load	Metrics and analytics displayed	Pass
T02	Events directory	Five sample events displayed	Pass
T03	Create event form	Required event fields available	Pass
T04	Registration log	Registration records display correctly	Pass
T05	Participant directory	Participant records and enrollments display	Pass
T06	Venue/schedule module	Venue and schedule allocations display	Pass
T07	Organizer/category module	Organizers and categories display	Pass
T08	Student portal	Upcoming events and seat availability display	Pass
T09	Schema console	Seven tables and relationships display	Pass
T10	SQL console	Query presets and execution area display	Pass
T11	Insert/delete evidence	Database change is reflected in UI/database	Pending final before/after evidence

14.1 Validation Rules

- Event capacity must be positive.
- Venue capacity must be positive.
- Registration fee cannot be negative.
- Schedule end time must be later than start time.
- Referenced category, organizer, event, venue and participant records must exist.

- A participant cannot register for the same event twice.
- Status and payment values are restricted to valid domain values.

15. Security, Integrity and Performance Considerations

15.1 Data Integrity

Referential integrity is maintained through foreign keys. Entity-level uniqueness and domain checks reduce inconsistent or duplicate data.

15.2 Input Validation

The UI forms should validate required fields, numeric capacity/fee values, email format and date/time values before executing database operations.

15.3 Indexing

Primary keys and UNIQUE fields are indexed automatically by the database. In a production deployment, additional indexes can be considered for event names, event dates, registration status and frequently searched email fields.

15.4 Transaction Awareness

Operations involving multiple related changes, such as registration followed by payment processing, can be wrapped in transactions so that partial updates do not leave inconsistent records.

16. Conclusion and Future Enhancements

The Event Registration and Venue Scheduling System successfully demonstrates how a relational DBMS can centralize event information and provide controlled access through a web interface. The final design connects categories and organizers to events, events to schedules and venues, and participants to events through registrations. The design is normalized and uses relational constraints to protect data integrity.

16.1 Future Enhancements

- Role-based authentication for administrators, organizers and students.
- Automatic venue-overlap detection before schedule creation.
- Automated email/SMS registration confirmations.
- Waitlists, cancellations, refunds and attendance tracking.
- Exportable PDF/CSV reports.
- More advanced analytics and trend dashboards.
- Production deployment with HTTPS, secure credentials and stronger access control.
- Database transactions and additional indexes for larger datasets.

17. References

- MySQL Documentation — <https://dev.mysql.com/doc/>

- PHP Manual — <https://www.php.net/docs.php>
- Apache Friends / XAMPP — <https://www.apachefriends.org/>
- diagrams.net — <https://app.diagrams.net/>
- GitHub Documentation — <https://docs.github.com/>

18. Contribution of Each Member

Member	Roll Number	Contribution
[Member 1 Name]	[Roll Number]	Database design, SQL, ER diagram and normalization
[Member 2 Name]	[Roll Number]	Backend/database connectivity, CRUD modules and testing
[Member 3 Name]	[Roll Number]	Frontend/UI, documentation, screenshots and presentation

Update this section to reflect the actual contribution and GitHub commit history of all three members.

19. Appendix: GitHub Repository

Repository: <https://github.com/Ananey-Sharma/DBMS-Course-Project>

- Presentation/
- Project-Report/
- README.md
- ER diagram
- SQL database script
- Source code
- UI screenshots